# TableFilter

A lightweight, client-side JavaScript library that adds **per-column filtering** to plain HTML tables.

TableFilter injects filter buttons into table headers, displays a Bootstrap modal for configuring filters (Text / Select / Multi-select / Range), persists filters in `sessionStorage`, and shows or hides table rows based on active filters.

This README is written to be **easy to understand for developers who just want to use the library**, while also being **detailed enough for developers who want to maintain or extend it**.

---

## ⚠ Important Initialization Rule

**TableFilter MUST be initialized only after the entire page (including the table) is fully rendered.**

If the library is called before the table exists in the DOM, header buttons and event handlers will not be attached correctly.

### Correct ways to initialize

#### 1. Place initialization at the bottom of the page

```
<script src="tablefilter.js"></script>
<script>
  TableFilter.applyTo('#myTable', {...});
</script>
</body>
```

#### 2. Use DOMContentLoaded

```
document.addEventListener('DOMContentLoaded', () => {
  TableFilter.applyTo('#myTable', {...});
});
```

#### 3. Use jQuery ready

```
$(function () {
  TableFilter.applyTo('#myTable', {...});
});
```

---

## Features

- Per-column filter button in table headers
- Filter types:
  - Text (contains, case-insensitive)
  - Select (single value)
  - Multi-select (multiple values)
  - Range (numbers or dates)

- Bootstrap modal UI
- Custom searchable select widget (no Select2 dependency)
- Filters persist using `sessionStorage`
- Zero server calls (client-side filtering)
- Designed for readability and extensibility

---

# Basic Usage

### HTML

```
<table id="students">
  <thead>
    <tr>
      <th>No</th>
      <th>Name</th>
      <th>Age</th>
      <th>Joined Date</th>
    </tr>
  </thead>
  <tbody>
    <tr><td>1</td><td>Alice</td><td>22</td><td>2023-01-15</td></tr>
    <tr><td>2</td><td>Bob</td><td>25</td><td>2022-11-10</td></tr>
  </tbody>
</table>
```

### JavaScript

```
TableFilter.applyTo('#students', {
  columnNames: ['No', 'Name', 'Age', 'Joined Date'],
  columnTypes: ['number', 'text', 'number', 'date']
});
```

---

# Configuration Options

| Option | Type | Description |
|---|---|---|
| `tableID` | `string` | Table DOM id (required when using constructor) |
| `columnNames` | `string[]` | Logical column names used to map headers |
| `columnTypes` | `string[]` | `text`, `number`, or `date` (default: `text`) |
| `storageKey` | `string` | Optional custom key for `sessionStorage` |

---

# Public API

### Static Method

```
TableFilter.applyTo(selector, options)
```

Creates a `TableFilter` instance for each table matching the selector.

- Each table **must** have an `id`.
- Prevents double initialization automatically.

---

### Constructor

```
new TableFilter({
  tableID,
  columnNames,
  columnTypes,
  storageKey
});
```

Creates a filter instance for a single table.

---

### Common Instance Methods

`applyAllFilters()`

Applies all active filters and returns the number of visible rows.

`clearCurrentFilterHandler()`

Clears the filter for the currently selected column.

`clearAllFiltersHandler()`

Clears all filters and shows all rows.

`saveFiltersToStorage()`
`loadFiltersFromStorage()`

Manually persist or restore filters.

---

# How Filtering Works (Internal Logic)

1. Each filter is stored in `currentFilters` using the **real table column index**.
2. When `applyAllFilters()` runs:
    - Each `<tbody><tr>` is evaluated.
    - Each active filter is applied to the relevant cell.
    - If any filter fails, the row is hidden (`display: none`).

### Filter Rules

- **Text** → case-insensitive `includes()`
- **Select** → strict equality
- **Multi-select** → value exists in selected list

- **Range**:
  - Dates → parsed as `Date` objects
  - Numbers → numeric comparison (commas removed)
  - Fallback → string comparison

---

# Storage Format

Filters are saved in `sessionStorage`:

```
{
  "2": {
    "type": "range",
    "column": 2,
    "columnType": "date",
    "columnName": "Joined Date",
    "from": "2023-01-01",
    "to": "2023-06-30",
    "timestamp": "2025-12-12T12:00:00Z"
  }
}
```

- Storage lifetime is **per browser tab**.
- Switch to `localStorage` for cross-session persistence.

---

# Advanced Internals (For Maintainers)

## Column Mapping

- Logical columns (`columnNames`) are mapped to real `<th>` indices.
- Header text matching prefers a `<span>` inside `<th>`.
- If header text doesn't match exactly, mapping fails.

## Debouncing

- Text and range inputs use a debounced apply (default ~300ms).
- A single `applyTimeout` is shared per instance.

## Custom Searchable Select

- Built by `makeSelectSearchable()`.
- Original `<select>` remains in DOM (hidden) for native compatibility.
- Custom UI supports:
  - Search input
  - Keyboard navigation
  - Multi-select tags
- API stored on `selectEl._sf` (`rebuildOptions`, `destroy`).

## Modal Safety

- Each instance owns its modal (`_instanceId`).
- Modal tracks its owning instance to prevent cross-instance conflicts.

---

## Dependencies

- **Required**: Bootstrap JS + jQuery (modal behavior)
- **Optional**: None

All filtering logic is pure vanilla JavaScript.

---

## Performance Notes

- Complexity: `O(rows × activeFilters)`
- Recommended for small-to-medium tables.
- For large datasets, consider server-side filtering.

---

## Common Issues

| Issue | Cause | Fix |
|---|---|---|
| Modal not opening | Bootstrap JS missing | Include Bootstrap & jQuery |
| Filters not matching columns | Header text mismatch | Wrap header text in `<span>` |
| Dates not filtering | Non-ISO date format | Normalize date strings |
| Filters lost on close | `sessionStorage` | Use `localStorage` instead |

---

## Extending the Library

Recommended extension points:

- Add callbacks in `applyFilterAutomatically()`
- Replace storage layer (`sessionStorage` → API / localStorage)
- Improve date parsing with `date-fns`
- Add `destroy()` method for full teardown
- Add ARIA attributes for accessibility

---

## License

## Final Notes

TableFilter is intentionally simple, readable, and flexible. It is ideal when you need filtering without heavy dependencies like DataTables, while still allowing future extension and customization.

If you need help extending or refactoring the library, this README and the inline code comments are designed to make that process straightforward.